Binary Search

"Searching"

→ Google Search
→ Amazon
→ Whatsapp
→ File in PC
→ Meaning of life
→ NauKri
→

unsorted data

CS

Linear Search

Data

Newspaper

Data

3	1	2	5	6

(5) Key / Target

Text

India Key

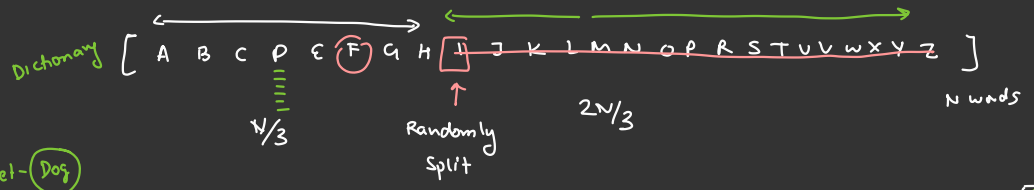
Dictionary [A ≡ B ≡ C ≡ D ≡ ----] Phonebooks

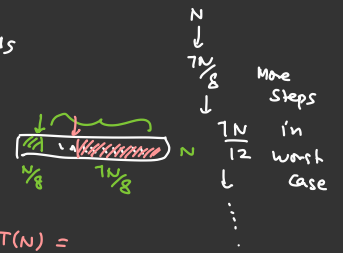
A —
B —
C —
⋮
Z —

Contact list

Rahul

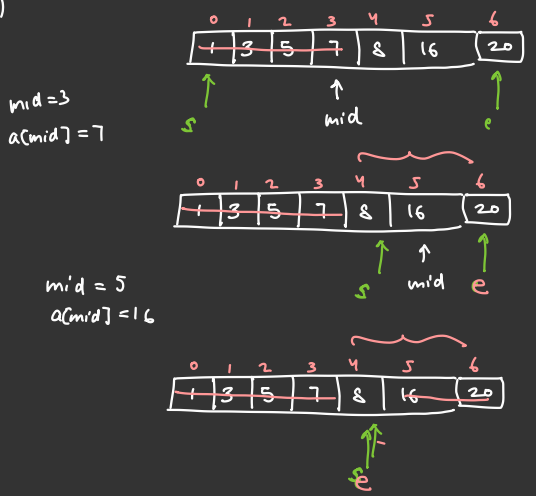
Binary Search $O(\log_2 N)$





Better option - Divide into two parts by finding out mid.
(Sorted data)

N=7

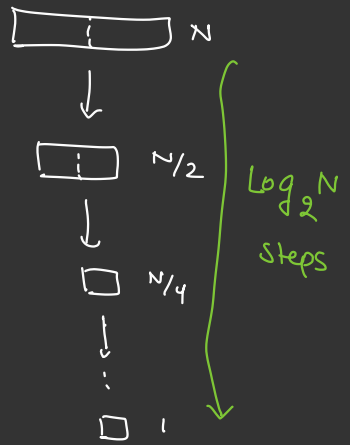


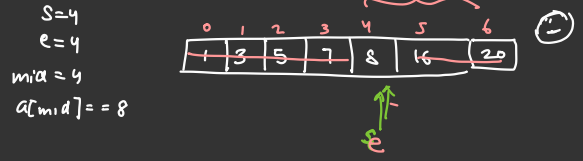
Search Space (0, n-1)

T = 8

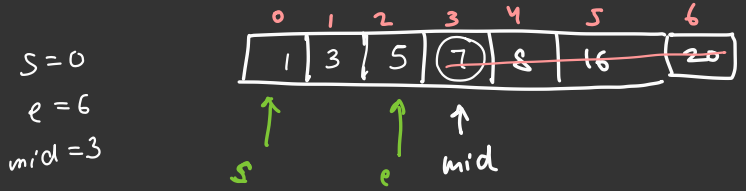
① T > a[mid]
s = mid+1

② T < a[mid]
e = mid - 1





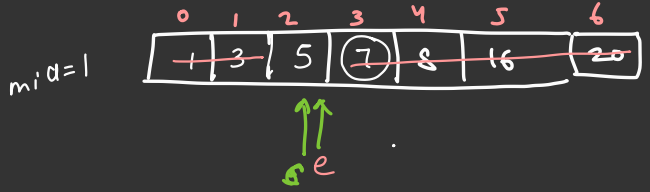
Case-II iF element is NoT present



T = 6

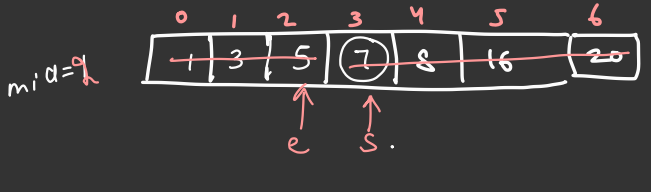
a[mid] > 6

e = mid - 1



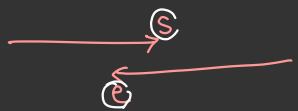
a[mid] < 6

s = mid + 1

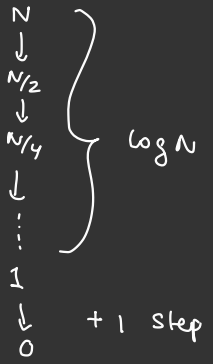


a[mid] < 6

s = mid + 1



s > e → return (-1) NOT present

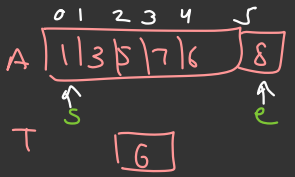


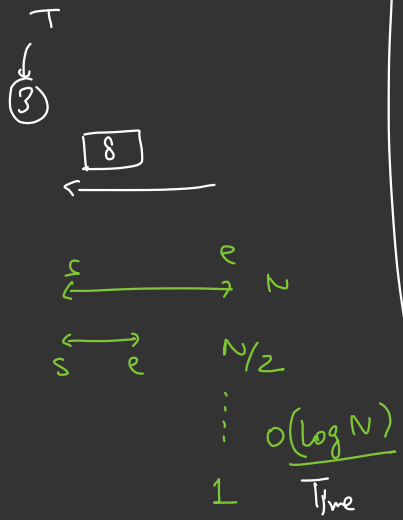
ToDo

3 Mins

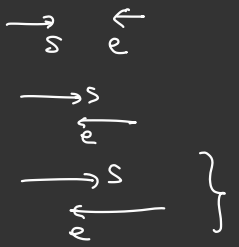
9:38

```
int binarySearch(A, T){
    s = 0
    e = A.length - 1
    while(s<=e){
        mid = (s+e)/2
        if(a[mid] == T)
            return mid
        else if (A[mid] > T)
            e = mid - 1
        else
            s = mid + 1
    }
    return -1;        // when s>e, come out of loop
}
```



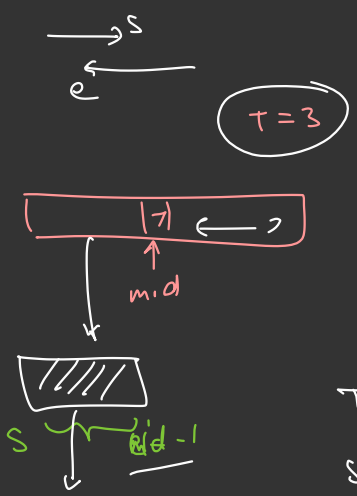


Space → O(1)



Binary Recursive Search

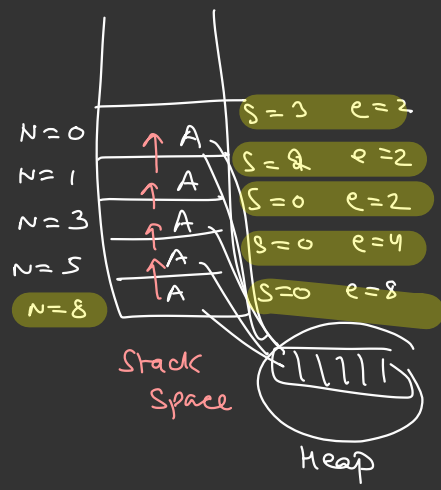
cost of extra space is there



Time : O(log N)

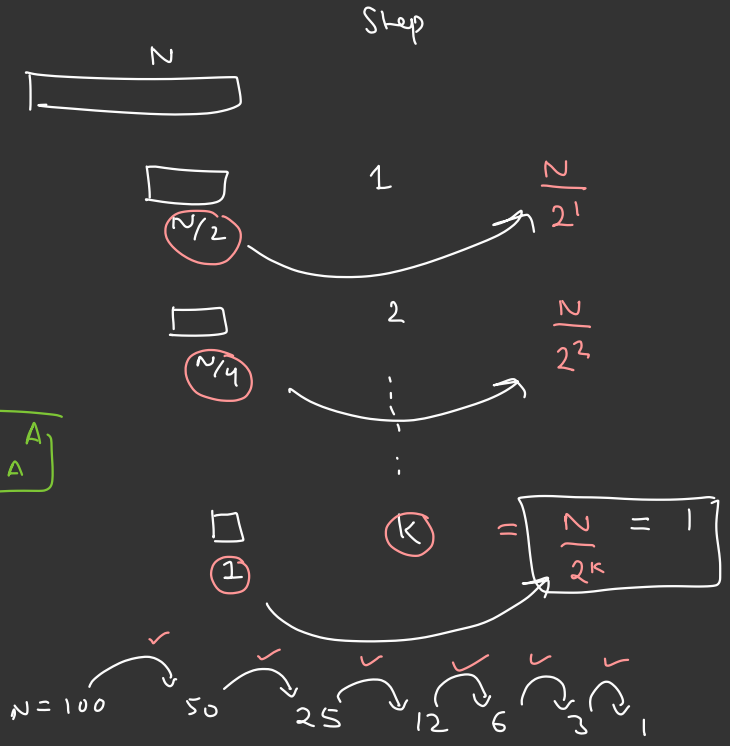
Space : O(log N)

```
int bSearch (A, T, S, e) {
  // Base Case
  if (s > e) {
      return -1;
  }
  // Rec Case
  mid = (s+e)/2
  if (A[mid] == T) {
      return mid
  }
  if (A[mid] > T) {
      return bSearch(A, T, S, mid-1)
  }
  else {
      return bSearch(A, T, mid+1, e)
  }
}
```



Time Complexity

Intuitively



$$\frac{N}{2^K} = 1$$

$$\Rightarrow 2^K = N$$

$$\Rightarrow \log_2 2^K = \log_2 N$$

$$\Rightarrow \boxed{K = \log N}$$

$$\log_A A^X = X \log_A A$$

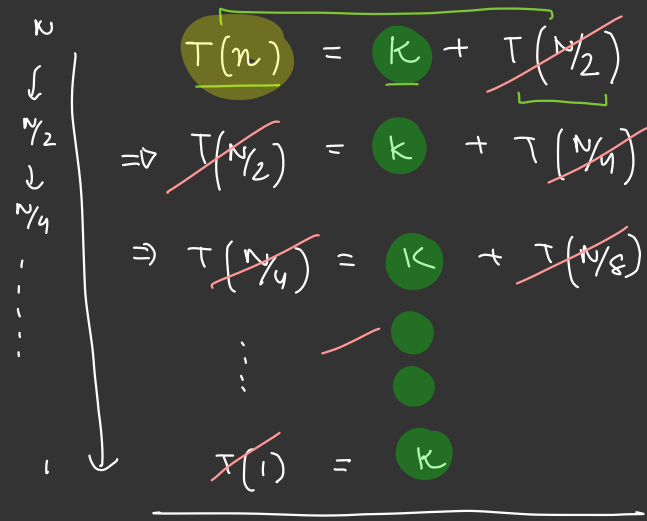
$$= \boxed{X}$$

$$\text{Log}_2 100 = 6.xx$$

$$= (6)$$

Mathematical way

(Substitution Method)



① k

② T(N/2)

$\log N$

$$T(n) = k + k + k + \cdots$$

$$= \sum_{\log N} k$$

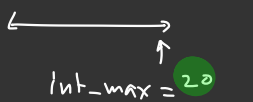
$$= k \cdot \log_2 N$$

$$= O(\log_2 N)$$

S e

↑ ↑

large + large => overflow ?



S = 12 e = 18

$mid = \frac{(s+e)}{2}$ => Replace $s + \frac{(e-s)}{2}$

$= \frac{2s + e - s}{2}$

$= \left(\frac{s+e}{2}\right)$

$\frac{(12+18)}{2} = \frac{\cancel{30}}{2}$ overflow

$\rightarrow \frac{12+18}{2} = \frac{30}{2} = 15$

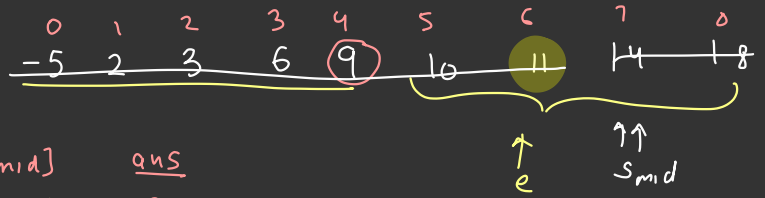
$\rightarrow 12 + \frac{(18-12)}{2} = 12 + \frac{6}{2}$

$= 15$

Hack optional, can be helpful sometimes

$mid = s + \frac{(e-s)}{2}$

Q. Given a sorted array of integers, find the greatest element <= Target in the array.



T = 12

	arr[mid]	ans
mid = 4	9 < 12	9
mid = 6	11 < 12	11
mid = 7	14 > 12	11
s > e	stop	

a[mid] < T
ans = arr[mid]
s = mid + 1

a[mid] == T
return mid

a[mid] > T
18 > 12
e = mid - 1
(discard Right)

Code

```
ans = INT_MIN
s = 0
e = A.length - 1

while (s <= e)
        mid = (s + e) / 2
        if (A[mid] == T)
                return mid
        else if (A[mid] < T){
                ans = A[mid]
                s = mid + 1
        }
        else {
                e = mid - 1
        }
}
return ans
```

Time → O(Log N)

Space → O(1)

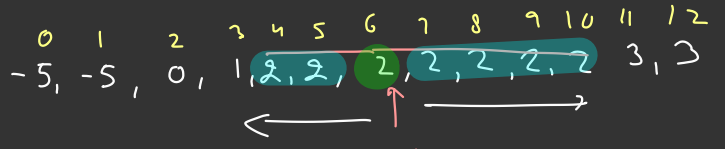
(a) Given a sorted array, Target

find frequency of (Target)

10 Min Break.

10 30

Example =>

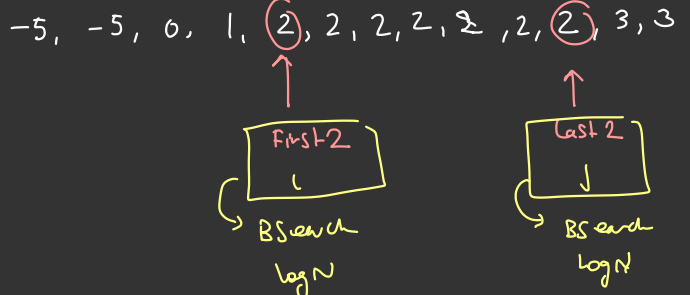


T = 2

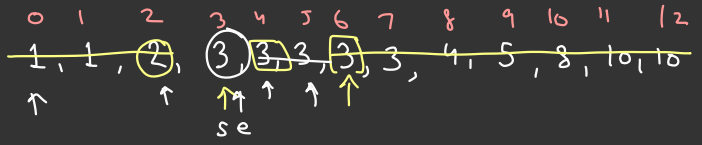
Output = (5)

Algo-1 O(N) or linear Search & Count

Algo-2



First Occ



T = 3

O(log N)

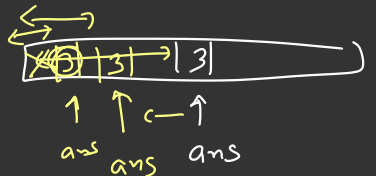
S = 0	e = 12	mid = 6	ans = 6	e = mid - 1	go left
S = 0	e = 5	mid = 2	———	S = mid + 1	go Right
S = 3	e = 5	mid = 4	ans = 4	e = mid - 1	go left
S = 3	e = 3	mid = 3	ans = 3	e = mid - 1	go left
e = 2	S = 3	Stop			

first occ.

Code

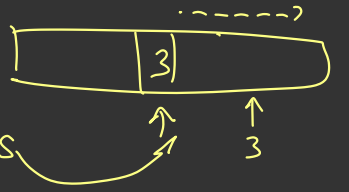
```
int firstOcc (A, T) {
    s = 0
    e = A.length - 1
    ans = -1
    while (s <= e) {
        mid = (s+e)/2
        if (a[mid] == T) {
            → ans = mid   ← go left ↓
            → e = mid - 1,
        }
        else if (a[mid] > T){
            e = mid - 1
        }
        else {
            s = mid + 1
        }
    }
    return ans.
}
```

mid



e = 3
s = 4
Stop

ans = ~~5~~ 4



3

O(Log N)

```
int lastOcc (A, T) {

    s = 0
    e = A.length - 1

    ans = -1

    while (s <= e) {

        mid = (s+e)/2

        if (a[mid] == T) {
            → ans = mid   ← go left ↓ go Right;
            → s = mid+1
        }
        else if (a[mid] > T){
            e = mid - 1
        }
        else {
            s = mid + 1
        }
    }
    return ans;
}
```

$freq_x$ = last occ − first occ + 1

$O(\log N)$

"Interesting Fact" : Library

C++

- binary Search
- lower bound
- Upper bound

$O(\log N)$

Timed Contests

- lower_bound(A, T) + upperbound(A, T) + 1

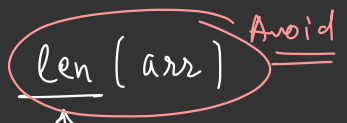
→ BinarySearch (----)

"Googling"

"Language Specific Class"

indexOf()
indexOf()

Sorted & Unsorted arrays → $O(N)$



O(N)

→ [arr.length]

↑ property of array object (Java)

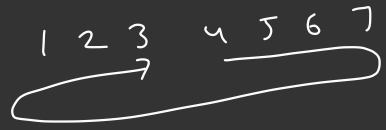
O(1) ←

O(1) → for length

```
binSearch (arr, N) :
    ==
    ==

main() {
    N = input() ←
    arr = [- - - - - ]
}
```

Q Sorted array rotated K times, in rotation we rotate element
Find out K / pivot element.

$K < len(A)$



Assume
No duplicates

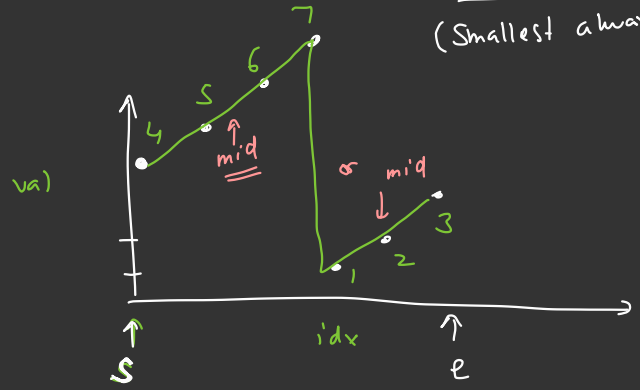
Input ⇒ [4 5 6 7 | 1 2 3]
0 1 2 3 4 5 6
0 1 2 3 0 1 2

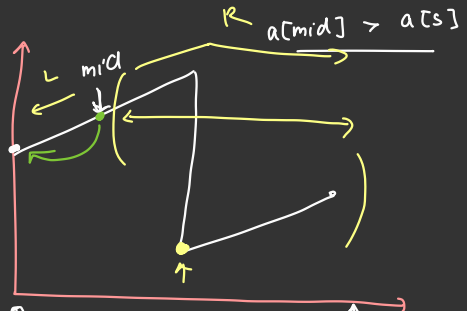
Pivot
(Smallest always)

Output

K = 4

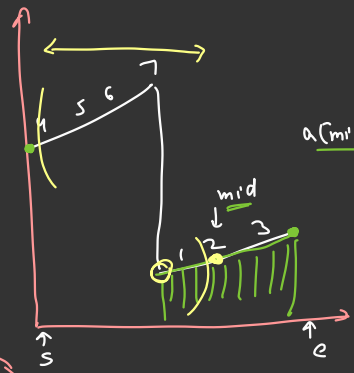
⇒ Not fully sorted





① go Right

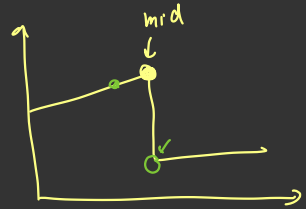
s = mid + 1



② go left

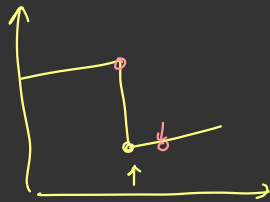
e = mid − 1

Corner Case



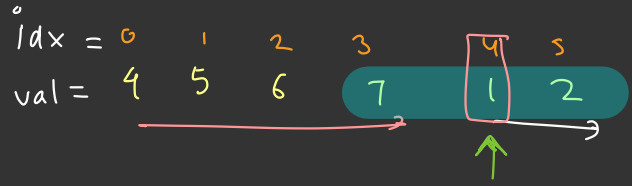
③ a[mid−1] < a[mid] > a[mid+1]

ans = mid + 1



④ a[mid−1] > a[mid] < a[mid+1]

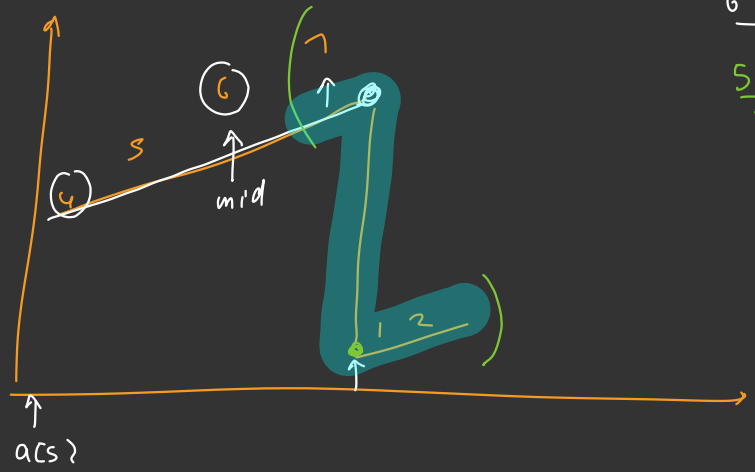
ans = mid



mid = 5/2 = 2

6 > 4 s = mid + 1

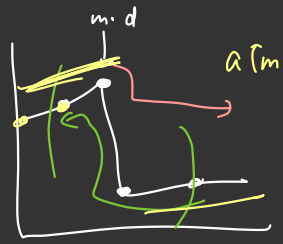
(5+3)/2 = 4



1 < 2 && 1 < 7

ans = mid

```
s = 0
e = n - 1

while ( s <= e ) {

        mid = (s+e)/2

        if ( mid<e && a[mid] > a[mid+1] )
                ans = mid + 1

        else if ( mid>s && a[mid] < a[mid-1] )
                ans = mid

        else if ( a[mid] > a[s] )
                s = mid + 1

        else
                e = mid - 1

}
```



mid = 0
mid - 1

```
    return ans;
```